

LangChain Learning Journey

A 4-Week Study Plan with Simple Code & Plain-English Explanations

Goal: Build production-ready chatbots and RAG apps

Level: Intermediate Python, new to LangChain

Pace: 1-2 hours a day, 4 weeks

Format: Explanation + runnable code for every topic

WEEK 1 · FOUNDATIONS → WEEK 4 · AGENTS & DEPLOYMENT

Week 1 — Foundations & Core Building Blocks

Milestone: a working chatbot that remembers conversation history.

DAY 1 Setup & LLM Basics

1.5h

LangChain is a framework that makes it easy to chain together LLM calls, prompts, tools, and data sources. Today is just plumbing: install the library, store your API key safely as an environment variable, and make your very first call to a model using `invoke()` — the standard method LangChain uses to "run" almost anything (a prompt, a chain, a model).

```
# Install (pick the provider you'll use)
pip install langchain langchain-openai langchain-anthropic python-dotenv

# .env file (never hardcode keys in code)
# OPENAI_API_KEY=sk-...
# ANTHROPIC_API_KEY=sk-ant-...
```

```
from dotenv import load_dotenv
from langchain_anthropic import ChatAnthropic

load_dotenv() # reads .env into environment variables

llm = ChatAnthropic(model="claude-sonnet-4-6")

response = llm.invoke("Explain LangChain in one sentence.")
print(response.content)
```

Key idea: Every LangChain component — models, prompts, parsers — shares the same interface: `.invoke()` runs it once, `.stream()` streams output, `.batch()` runs many inputs at once. Learn this pattern once, reuse it everywhere.

DAY 2 Prompt Templates

1h

Hardcoding prompts as strings gets messy fast. `PromptTemplate` and `ChatPromptTemplate` let you define a prompt with placeholder variables, then fill them in at runtime — like an f-string, but composable and reusable across your app. Few-shot prompting means showing the model a couple of example input/output pairs before asking your real question, which improves consistency.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a concise Python tutor."),
    ("human", "Explain {topic} in {num_sentences} sentences."),
])

# .invoke() fills the placeholders and returns formatted messages
messages = prompt.invoke({"topic": "decorators", "num_sentences": 2})
print(messages.to_messages())
```

Key idea: A prompt template on its own is not a chain — it just formats text. You'll connect it to a model tomorrow with LCEL.

DAY 3 LCEL (LangChain Expression Language)

1.5h

LCEL is LangChain's way of chaining components together using the pipe operator `|`, similar to Unix pipes. Every chainable object (prompt, model, parser, retriever) is a "Runnable." Piping them means "take the output of the thing on the left and feed it as input to the thing on the right."

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_anthropic import ChatAnthropic

prompt = ChatPromptTemplate.from_template("Give one fun fact about {animal}.")
llm = ChatAnthropic(model="claude-sonnet-4-6")
parser = StrOutputParser()

# The chain: prompt -> model -> parser
chain = prompt | llm | parser

print(chain.invoke({"animal": "octopus"}))
```

Key idea: Because every piece implements the same Runnable interface, you can swap models, add steps, or reorder the chain without rewriting glue code.

Models return raw text by default. Output parsers convert that text into something your code can use directly — a plain string, or a validated structured object. `StrOutputParser` just extracts the text; `PydanticOutputParser` forces the model's output into a Pydantic schema so you get typed, structured data (great for extracting fields like name/date/amount).

```
from pydantic import BaseModel, Field
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_anthropic import ChatAnthropic

class Recipe(BaseModel):
    name: str = Field(description="dish name")
    ingredients: list[str]

parser = PydanticOutputParser(pydantic_object=Recipe)

prompt = ChatPromptTemplate.from_template(
    "Suggest a recipe for {craving}.\n{format_instructions}"
).partial(format_instructions=parser.get_format_instructions())

chain = prompt | ChatAnthropic(model="claude-sonnet-4-6") | parser
recipe = chain.invoke({"craving": "something spicy"})
print(recipe.name, recipe.ingredients)
```

Gotcha: Newer LangChain versions favor `llm.with_structured_output(Recipe)` over `PydanticOutputParser` — it's more reliable because it uses native tool-calling under the hood. Try both and see which your provider supports.

LLMs are stateless — each call knows nothing about previous calls. "Memory" in LangChain just means storing past messages and re-sending them with every new request. The modern approach uses `RunnableWithMessageHistory` (older `ConversationBufferMemory` classes are legacy but still widely referenced in tutorials).

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.prompts import ChatPromptTemplate
from langchain_anthropic import ChatAnthropic

store = {} # session_id -> history
def get_history(session_id):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("placeholder", "{history}"),
    ("human", "{input}"),
])
chain = prompt | ChatAnthropic(model="claude-sonnet-4-6")

chat = RunnableWithMessageHistory(chain, get_history,
    input_messages_key="input", history_messages_key="history")

cfg = {"configurable": {"session_id": "user-1"}}
chat.invoke({"input": "My name is Sam."}, config=cfg)
print(chat.invoke({"input": "What's my name?"}, config=cfg).content)
```

Key idea: "Memory" is just history you manage and re-inject — there's no magic persistence unless you store it (e.g. in a database) yourself.

DAY 6 Mini Project — Simple Chatbot

1h

Combine Days 1-5: a prompt template, a model, and message history, wrapped in a loop that keeps the conversation going. No retrieval yet — this is pure conversational memory.

```
while True:
    user_input = input("You: ")
    if user_input.lower() in ("exit", "quit"):
        break
    reply = chat.invoke({"input": user_input}, config=cfg)
    print("Bot:", reply.content)
```

✓ **Milestone reached:** a chatbot that remembers what you told it earlier in the conversation.

DAY 7 Review & Recap

0.5h

No new code — consolidate what you learned this week.

Self-check before Week 2:

- Can you explain what `.invoke()` does without looking it up?
- Can you write an LCEL chain (`prompt` | `llm` | `parser`) from memory?
- Do you understand why memory has to be manually re-injected into every call?
- Rebuild Day 6's chatbot from scratch, without copying your old code.

Week 2 — Retrieval-Augmented Generation (RAG) Core

Milestone: a working RAG pipeline answering questions over your own documents.

DAY 8 Document Loaders

1.5h

RAG starts with getting your own data into a format LangChain understands: a list of `Document` objects, each with `page_content` (text) and `metadata` (like source filename). Loaders exist for PDFs, web pages, text files, Notion, Slack, and dozens more sources.

```
pip install pypdf langchain-community
```

```
from langchain_community.document_loaders import PyPDFLoader, WebBaseLoader

pdf_docs = PyPDFLoader("handbook.pdf").load()
web_docs = WebBaseLoader("https://example.com/article").load()

print(pdf_docs[0].page_content[:200])
print(pdf_docs[0].metadata)  # e.g. {'source': 'handbook.pdf', 'page': 0}
```

DAY 9 Text Splitting

1.5h

A whole PDF is too big to hand to an embedding model or fit in a prompt. You split documents into smaller "chunks." `chunk_size` controls chunk length; `chunk_overlap` repeats a little text between chunks so context isn't lost at the boundary. Too small and you lose context; too large and retrieval gets imprecise — this is a tuning knob you'll revisit often.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=150,
)
chunks = splitter.split_documents(pdf_docs)
print(len(chunks), "chunks created")
```

Key idea: "Recursive" means it tries to split on paragraph breaks first, then sentences, then words — so chunks stay as semantically coherent as possible.

DAY 10 Embeddings

1.5h

An embedding model turns text into a list of numbers (a vector) that captures its meaning. Texts with similar meaning end up as vectors that are close together mathematically. This is what makes "search by meaning" (not just keyword matching) possible.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector = embeddings.embed_query("How do I reset my password?")
print(len(vector), vector[:5])  # e.g. 1536 numbers, first 5 shown
```

Key idea: You never read embeddings yourself — a vector store compares them for you using distance math (cosine similarity, etc.) to find the closest matches.

DAY 11 Vector Stores

2h

A vector store is a database specialized for storing embeddings and doing fast similarity search. Chroma and FAISS both run locally with no external service — perfect for learning before you touch a hosted option in Week 3.

```
pip install langchain-chroma chromadb
```

```
from langchain_chroma import Chroma

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db",
)

results = vectorstore.similarity_search("password reset steps", k=3)
for r in results:
    print(r.page_content[:150], "---")
```

DAY 12 Retrievers

1.5h

A retriever wraps a vector store behind a standard Runnable interface so it plugs into LCEL chains. Beyond plain similarity search, MMR (Maximal Marginal Relevance) balances relevance with diversity, avoiding five near-duplicate chunks for one query.

```
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 4, "fetch_k": 20},
)

docs = retriever.invoke("How do I reset my password?")
print([d.metadata.get("source") for d in docs])
```

DAY 13 Basic RAG Chain

2h

Now wire it together: retrieve relevant chunks, stuff them into a prompt as context, and ask the model to answer using only that context. This "retrieve then generate" pattern is the heart of RAG.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("""Answer using only this context.
If the answer isn't in the context, say you don't know.

Context:
{context}

Question: {question}""")

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | ChatAnthropic(model="claude-sonnet-4-6")
    | StrOutputParser()
)

print(rag_chain.invoke("How do I reset my password?"))
```

✓ **Milestone reached:** a working RAG pipeline answering questions over your own documents.

Rebuild the RAG chain end-to-end without looking at your notes.

- Load → split → embed → store → retrieve → prompt → generate: can you list these steps unprompted?
- Try changing `chunk_size` and re-running — notice how answers change.
- Print the retrieved chunks before generation to sanity-check retrieval quality separately from generation quality.

Week 3 — Production RAG & Chatbot Patterns

Milestone: a conversational RAG chatbot with memory + streaming.

DAY 15 Conversational RAG

1.5h

Plain RAG breaks on follow-up questions like "what about the second one?" because the retriever doesn't know what "the second one" refers to. A history-aware retriever first rewrites the question using chat history, then retrieves — so retrieval understands context, not just the literal last message.

```
from langchain.chains import create_history_aware_retriever
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

contextualize_prompt = ChatPromptTemplate.from_messages([
    ("system", "Rewrite the user's question as a standalone question, using chat history."),
    MessagesPlaceholder("chat_history"),
    ("human", "{input}"),
])

history_aware_retriever = create_history_aware_retriever(
    ChatAnthropic(model="claude-sonnet-4-6"), retriever, contextualize_prompt
)
```

Key idea: This is an extra LLM call before retrieval — it costs a little latency, but massively improves multi-turn accuracy.

DAY 16 Advanced Retrieval

1.5h

Plain similarity search misses relevant chunks phrased differently than the query. Multi-query retrieval asks the LLM to generate several rephrasings of the question and unions the results. Contextual compression re-ranks and trims retrieved chunks down to just the relevant sentences before they reach the prompt.

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

mq_retriever = MultiQueryRetriever.from_llm(
    retriever=retriever, llm=ChatAnthropic(model="claude-sonnet-4-6")
)

compressor = LLMChainExtractor.from_llm(ChatAnthropic(model="claude-sonnet-4-6"))
compressed_retriever = ContextualCompressionRetriever(
    base_compressor=compressor, base_retriever=mq_retriever
)
```

Gotcha: These patterns add extra LLM calls per query — great for accuracy, costly for latency/spend at scale. Measure before shipping to production.

DAY 17 Vector DB Upgrade

2h

Local stores like Chroma/FAISS don't scale to production traffic or multiple servers. Hosted vector DBs (Pinecone, Qdrant, Weaviate) add persistence, scaling, filtering, and metadata search. The LangChain interface stays nearly identical — swapping stores is mostly a one-line change.

```
pip install langchain-pinecone pinecone-client
```

```
from langchain_pinecone import PineconeVectorStore

vectorstore = PineconeVectorStore.from_documents(
    documents=chunks,
    embedding=embeddings,
    index_name="my-rag-index",
)

retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
```

DAY 18 Streaming Responses

1.5h

Waiting for a full response before showing anything feels slow. Streaming sends tokens to the UI as the model generates them — the same technique behind ChatGPT's "typing" effect. Every LCEL chain supports `.stream()` automatically.

```
for chunk in rag_chain.stream("How do I reset my password?"):
    print(chunk, end="", flush=True)
```

Key idea: If your chain ends in `StrOutputParser()`, `.stream()` yields text chunks directly — no extra parsing needed on the frontend.

DAY 19 Evaluation Basics

1.5h

"It looks right" isn't a test. LangSmith traces every chain run and lets you build a small dataset of question/expected-answer pairs to score your RAG pipeline automatically, catching regressions when you change chunk size, prompts, or models.

```
pip install langsmith
# .env: LANGSMITH_API_KEY=..., LANGSMITH_TRACING=true
```

```
from langsmith import Client
from langsmith.evaluation import evaluate

client = Client()
dataset = client.create_dataset("rag-qa-eval")
client.create_examples(
    inputs=[{"question": "How do I reset my password?"}],
    outputs=[{"answer": "Go to Settings > Security > Reset Password."}],
    dataset_id=dataset.id,
)

def target(inputs):
    return {"answer": rag_chain.invoke(inputs["question"])}

evaluate(target, data="rag-qa-eval", evaluators=["correctness"])
```

DAY 20 Project Work

2h

Apply everything so far to a real dataset — your own notes, a company's public docs, or a set of Wikipedia pages. No new concept today; the goal is friction. You'll hit real messy-data problems (bad PDF text extraction, huge documents, irrelevant retrieval) that tutorials don't show you.

- Pick a real, slightly messy data source (not a clean toy example).
- Load → split → embed → store → build a conversational retriever chain.
- Ask it 10 real questions and note where it fails.

DAY 21 Review & Recap

1h

Debug and refine yesterday's project rather than learning something new.

- For any bad answer, check: was it a retrieval failure (wrong chunks) or a generation failure (right chunks, bad answer)?
- Try MMR vs plain similarity search and compare results.

✓ **Milestone reached:** a conversational RAG chatbot with memory + streaming.

Week 4 — Agents, Deployment & Polish

Milestone: a deployed, polished RAG chatbot you can show in a portfolio.

DAY 22 Intro to Agents

1.5h

A chain always runs the same fixed steps. An agent lets the LLM decide which steps to take, and in what order, by choosing from a set of "tools" (functions it can call). `create_react_agent` implements the "ReAct" pattern: the model reasons about what to do, calls a tool, observes the result, and repeats until it can answer.

```
from langchain_core.tools import tool
from langgraph.prebuilt import create_react_agent

@tool
def get_weather(city: str) -> str:
    """Get current weather for a city."""
    return f"{city} is sunny and 24°C"

agent = create_react_agent(ChatAnthropic(model="claude-sonnet-4-6"), tools=[get_weather])
result = agent.invoke({"messages": [("human", "What's the weather in Kathmandu?")]})
print(result["messages"][-1].content)
```

DAY 23 Custom Tools

1.5h

Turn your Week 2 retriever into a tool so an agent can decide, on its own, when to search your documents versus just answering from general knowledge. Any Python function can become a tool with the `@tool` decorator — the docstring becomes the description the LLM reads to decide whether to use it.

```
@tool
def search_docs(query: str) -> str:
    """Search internal documentation for relevant information."""
    docs = retriever.invoke(query)
    return "\n\n".join(d.page_content for d in docs)

agent = create_react_agent(ChatAnthropic(model="claude-sonnet-4-6"), tools=[search_docs])
```

DAY 24 LangGraph Intro

1.5h

LCEL chains are linear pipelines — no loops, no branching based on conditions. LangGraph models your application as a graph of nodes (steps) and edges (transitions), which is what you need for agents that loop, retry, or branch. It's LangChain's answer to "chains aren't flexible enough for complex apps."

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict):
    question: str
    answer: str

def answer_node(state: State) -> State:
    state["answer"] = rag_chain.invoke(state["question"])
    return state

graph = StateGraph(State)
graph.add_node("answer", answer_node)
graph.set_entry_point("answer")
graph.add_edge("answer", END)

app = graph.compile()
print(app.invoke({"question": "How do I reset my password?"}))
```

Key idea: Start simple — this trivial one-node graph does what a chain already does. The payoff comes when you add conditional edges (e.g. "if the answer is uncertain, loop back and retrieve again").

DAY 25 Error Handling

1.5h

Production apps need to survive rate limits, timeouts, and model refusals gracefully. LangChain chains support `.with_retry()` for transient errors and `.with_fallbacks()` to fall back to a different model or a canned response when the primary path fails.

```
robust_chain = rag_chain.with_retry(
    stop_after_attempt=3,
).with_fallbacks([
    RunnableLambda(lambda _: "Sorry, I'm having trouble right now — please try again shortly.")
])
```

Gotcha: A fallback that swallows every error can hide real bugs. Log the original exception even when you fall back gracefully.

DAY 26 Deployment

2h

Wrap your chain in a web framework so it's actually usable by someone other than you. FastAPI is the common choice for a real backend with streaming support; Streamlit is faster for a throwaway internal demo UI.

```
pip install fastapi uvicorn
```

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.post("/chat")
def chat(question: str):
    def generate():
        for chunk in rag_chain.stream(question):
            yield chunk
    return StreamingResponse(generate(), media_type="text/plain")

# Run with: uvicorn main:app --reload
```

DAY 27 Final Project Polish

2h

A trustworthy RAG app shows its sources. Return the retrieved document metadata alongside the answer so users can verify claims — this single feature is what separates a demo from something people will actually trust.

```
def answer_with_sources(question: str):
    docs = retriever.invoke(question)
    answer = rag_chain.invoke(question)
    sources = [d.metadata.get("source", "unknown") for d in docs]
    return {"answer": answer, "sources": list(set(sources))}

print(answer_with_sources("How do I reset my password?"))
```

DAY 28 Wrap-Up

1h

No code today — document what you built.

README checklist:

- What problem does the app solve, and what data does it answer questions over?
- Architecture diagram or bullet list: loader → splitter → embeddings → vector store → retriever → agent/chain → API.
- Known limitations and what you'd improve with more time.
- What broke along the way, and what you learned fixing it (your "gotchas" file).

✓ **Final milestone:** a deployed, polished RAG chatbot you can show in a portfolio.

Resources & Tips for Retention

Resource	Use it for
python.langchain.com	Official docs — always check the version, the API changes fast
LangChain Academy	Free official structured courses
LangSmith	Tracing and debugging chains, and evaluation (Day 19)
Your own notes / a company's public docs / Wikipedia	Realistic practice datasets for RAG

- Tips for retention:**
- Rebuild each mini-project from memory before moving to the next week — recall beats re-reading.
 - Keep a running "gotchas" file: LangChain's API changes often, so note what broke and why.
 - Prefer LCEL syntax (`|>`) over legacy `Chain` classes like `LLMChain` — LCEL is the modern standard and what current docs assume.